

Neil Chulani
Prof. Harris
System On Chip Design
28 April, 2025

E154 Final Project

Floating-Point Multiply-Accumulate (FMA) Design

[Git Repository](#)

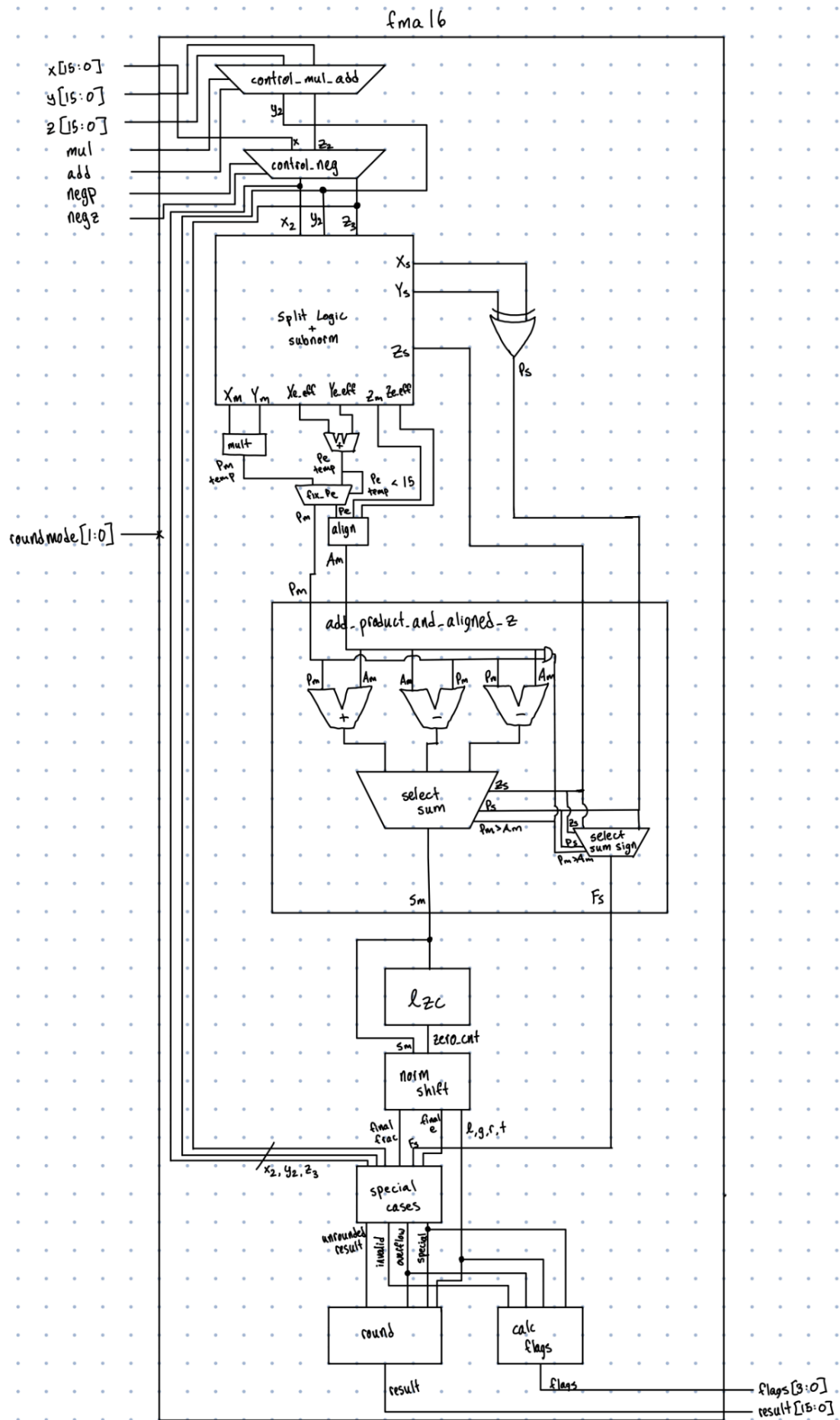
Introduction

The goal of this project was to design and implement an optimized half precision FMA unit to work with IEEE standards and RISC-V cores in order to enable low power computer graphics. On a high level, the FMA accepts three half precision floats X , Y , and Z , and computes $W = XY + Z$. The following implementation handles normalized numbers, subnormal inputs, special cases, and 4 rounding modes, as well as produces invalid, overflow, and inexact flags.

This FMA design works through the following 9 main steps:

1. Use the control signals to construct 'real' inputs to the FMA
2. Split the 'real' inputs into their signs, exponents, and create their mantissas
3. Multiply X and Y to get P (product)
4. Align Z with P , call this A (aligned)
5. Add P and A to get S (sum)
6. Use a priority encoder to detect leading 0s and normalize S
7. Check for special cases
8. Round
9. Set flags based on special cases and normalization

A block diagram of the design, with the steps flowing from top to bottom, can be seen below.



One of the notable optimizations about this design is how the round and flag stages take place in parallel. This allows for the system to compute the final result and the flags in a quicker manner since these pieces are not dependent on each other. The design also uses a leading zero counter, with inspiration taken from the Wally core, to compute the number of leading zeros in the sum to help perform normalization shifting. Before the use of this LZC, I designed two approaches, first using a simple for loop, and the second using a binary search-esque method for finding the first 1. The LZC did end up being the most efficient option for both area and timing, which is why it is used in the final design.

My first (and main) implementation of this design used a single fma16 module with all of the logic contained in the same level. The code structure remained clear, especially with rigorous comments and the accompanying block diagram, but one downside to this was the difficulty in understanding synthesis results, which will be discussed later. That being said, I did implement the same design, but splitting each step/block into its own module. This change in implementation increased the area and the critical path of the design by a non-insignificant amount. As such, the final design does not use extra modules for code structure and instead remains as one thoroughly commented top level module.

Test Results

This design is lint clean and passes all required tests, including just under 1 million tests generated by fma_testgen.c, as well as all baby_torture and torture tests. On top of this, the design passes tests that contain subnormal inputs, like 0001_7753_3C00_0C_3C01_1, a test where X has an exponent with all 0s. In this specific case, we have $5.97 \cdot 10^{-8} \cdot 30000 + 1 = 1.001 = 3C01$ as expected, but if the design didn't handle subnorm inputs, $0001 = 3.055 \cdot 10^{-5}$, which would result in $1.9165 = 3FAA$. So, even though this wasn't a requirement for the project, this added functionality is present (and was a very useful exercise). The user defined test cases included those produced by fma_testgen.c, which contained a range of relatively simple calculations, to some with each of the different types of rounding, and even ones with subnormal inputs. This all culminated in a test vector file of 86MB, and a phenomenal testbench display message: "932716 tests completed with 0 errors." Below are screenshots of lint, fma_testgen, baby_torture, and torture results.

```
(cvw) [inchulani@vlsi fma16]$ ./lint-fma
- V e r i l a t i o n   R e p o r t: Verilator 5.033 devel rev v5.032-61-ge0e164cea
- Verilator: Built from 0.030 MB sources in 3 modules, into 0.040 MB in 5 C++ files needing 0.000 MB
- Verilator: Walltime 0.009 s (elab=0.001, cvt=0.004, bld=0.000); cpu 0.009 s on 1 threads; allocated 8.980 MB
fma16 passed lint
```

```
# Loading work.lzc(fast)
#      932716 tests completed with          0 errors
# ** Note: $stop      : testbench.sv(49)
#      Time: 9327175 ns  Iteration: 1  Instance: /testbench_fma16
# Break in Module testbench_fma16 at testbench.sv line 49
# Stopped at testbench.sv line 49
VSIM 2> █
```

```
# Loading work.lzc(fast)
#      3624 tests completed with          0 errors
# ** Note: $stop      : testbench.sv(49)
#      Time: 36255 ns   Iteration: 1  Instance: /testbench_fma16
# Break in Module testbench_fma16 at testbench.sv line 49
# Stopped at testbench.sv line 49
VSIM 2> █
```

```
# Loading work.lzc(fast)
#      73331 tests completed with          0 errors
# ** Note: $stop      : testbench.sv(49)
#      Time: 733325 ns  Iteration: 1  Instance: /testbench_fma16
# Break in Module testbench_fma16 at testbench.sv line 49
# Stopped at testbench.sv line 49
VSIM 2> █
```

Synthesis Results

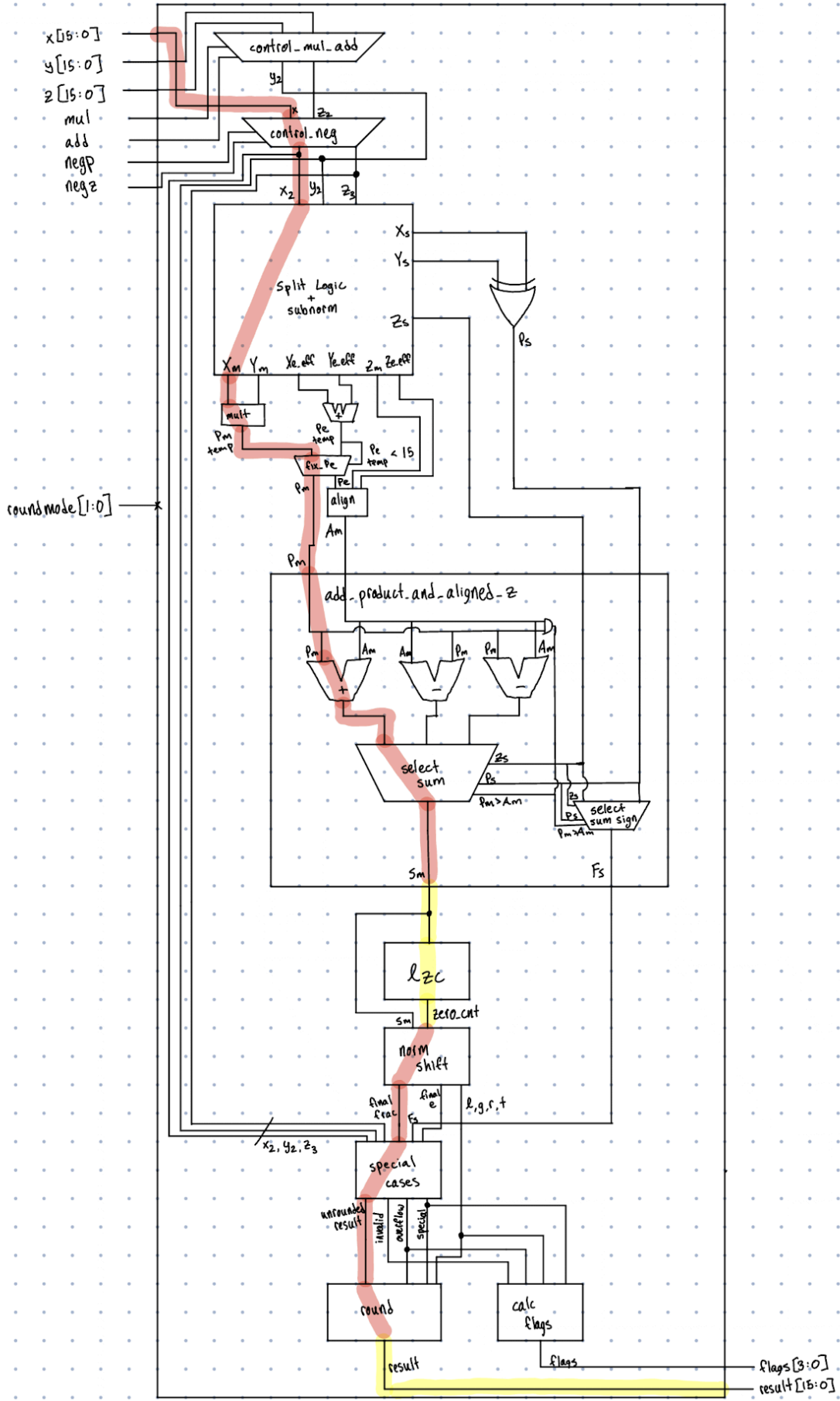
When synthesizing the main design, which only has one submodule for the LZC, the area report contained information that the total cell area is 40,448 μm^2 , but since there weren't many submodules, the hierarchical area distribution was not extremely helpful. All that it provided is that the fma16 module made up 94.2% of the wrapped design while the priority encoder, which was LZC from Wally, took up 5.6% of the design. This is largely unhelpful in understanding the area of this FMA implementation, so I restructured the code to use modules for each of the steps/blocks in the design. As expected, upon synthesis, this resulted in a similar area, but a much more informative hierarchical area distribution:

Hierarchical area distribution						
Hierarchical cell	Global cell area		Local cell area			
	Absolute Total	Percent Total	Combi-national	Noncombi-national	Black-boxes	Design
fma16wrapper	45731.8219	100.0	8.7912	2353.1113	0.0000	fma16wrapper
fma16	43369.9193	94.8	767.7648	0.0000	0.0000	fma16
fma16/add_p_z	15137.4694	33.1	15137.4694	0.0000	0.0000	add_product_and_aligned_z
fma16/align	3661.0464	8.0	3661.0464	0.0000	0.0000	align_z
fma16/compute_flags	52.7472	0.1	52.7472	0.0000	0.0000	flag_unit
fma16/hcs_i	519.1692	1.1	519.1692	0.0000	0.0000	handle_control_signals
fma16/multiply	9709.3919	21.2	9709.3919	0.0000	0.0000	compute_product
fma16/norm_shift	11168.7310	24.4	8692.0546	0.0000	0.0000	normalize
fma16/norm_shift/priority_enc	2476.6763	5.4	2476.6763	0.0000	0.0000	lzc
fma16/round	1513.0632	3.3	1513.0632	0.0000	0.0000	rounding_unit
fma16/specialcases	840.5364	1.8	840.5364	0.0000	0.0000	special_cases
Total			43378.7105	2353.1113	0.0000	

From this data, we can see that the block taking up the most area is the adding of the product and z. This is reasonable since the addition is the most hardware intensive portion of the FMA process. The two other large area modules are the multiplication and the normalization blocks, both of which are reasonably large parts of the design. It is important to note that the area report in the above picture is not the implementation considered to be the true final implementation, but it is the same design that allows for easier analysis. So, the true final total area is still 40,448 μm^2 as previously stated.

For the timing report, we get a critical path length of 6.424471ns. When using the final implementation, similar to the area report, the use of only one main module restricts the amount of information given in the timing report. That being said, we can see the LZC as well as the final result as part of the critical path, which allows us to reconstruct the critical path as follows:

fma16



Since we know that the LZC is in the critical path, the output, which leads to norm shift must be as well. From the normshift block, we must go to the special cases. Then from special cases, since we know we must end at result, the critical path goes through the round block. Working backwards from LZC, the sum must be in the critical path and thus one of the adders. These take in the aligned Am as input, which comes from the fixPe and mult blocks. Continuing backwards, we must have the split logic and control signals. While this is a reasonable reconstruction, we can also use the modular implementation to verify this. The modular implementation's timing report, while it ends up with a different critical path length, likely shares the same critical path, which is as follows:

Input → handle control signals	0.349812
Handle control signals → compute product	0.489179
Compute product → Add product and align z	3.010953
Add product and align z → normalize (and lzc)	4.144441
Normalize (and lzc) → special cases	5.616993
Special cases → rounding unit	6.016680
Rounding unit → result	6.649857

With this data, we can not only gain confidence in the reconstruction of the final implementation's critical path, but also understand that the steps that contribute the most to the critical path are the multiplication (compute product), adding, and normalizing. Similar to the area report, the above timing is not that of the final implementation and was just used to understand the design, so the final critical path remains 6.424471ns.

Using our total area and critical path, we get an ADP of 259,858.9993224297 $\mu\text{m}^2 \cdot \text{ns}$, which is approximately .259 $\text{mm}^2 \cdot \text{ns}$. There are two main reasons that this ADP would be higher than other implementations for this project. Firstly, this design carries 70 bits through the adding and normalization logic to allow for a easily understandable and robust implementation, which is demonstrated by the 100% torture test case coverage. That being said, it would be more efficient to discard bits that do not affect the result and calculate the sticky bit earlier on (during alignment of the addend to be specific), but this remains unimplemented in this design. Another reason for a larger ADP when compared to other implementations for this project is that the design does contain more functionality with the subnormal input handling, which clearly increases the area due to added hardware, but also increases the delay since the subnormal handling falls on the critical path. This is a very reasonable increase in ADP since it handles a more thorough set of cases than even torture addresses (though it was technically out of scope of the project - but it was well worth it to work on!).

Project Management

The following table contains the weekly breakdown of implementation progress and the time spent doing so:

Weekly Progress	Time Spent
Simple Multiplication	2 hours
All Multiplication	3 hours
All Addition (and some FMA)	7 hours
All FMA	2 hours
Special Cases (and subnorm)	6 hours
Rounding, Optimization, Debugging	10 hours
Final Optimization	1 hour
Total:	31 hours

The biggest thing that I'll carry from this project into future hardware design projects was the use of exhaustive test vectors. I've never tested one of my designs so thoroughly before (partly because so many edge cases weren't present, but partly because of a lack of a final torture test). Having a well defined testbench with a test vector generation script was an integral part of debugging and developing my FMA.

Another helpful thing that I noticed while designing subsequent parts of my FMA was having a block diagram ready prior to implementing any code. Throughout MicroPs and other digital design projects I've done, the block diagram was always an afterthought. By the later weeks, each change I made to the design started as a change to the block diagram, which was followed by a change to the code. This process helped me ensure that the design seemed reasonable before having to deal with implementation difficulties.

Conclusion

As a Computer Science and Mathematics double major from Pomona, digital design is a passion of mine that has mostly been self taught and is largely driven by a desire to understand every possible detail of a computer. Thus, having not spent a ton of time with large verilog designs, I greatly benefited from this project and really enjoyed the process of synthesizing and optimizing my own FMA. One of the biggest things I began to recognize throughout the process was just how much small implementation changes, like the declaration of a registered value to avoid recomputation, can impact the area and timing of a design. In fact, while it wasn't in my design, in Wally's, recomputing P_e when using it for alignment purposes actually speeds up the design since otherwise P_e falls in the critical path. Getting to see such intricate details that move past assembly level optimization, which I've spent lots of time with, was extremely rewarding. Taking this class is greatly adding to my desire to continue down a path of higher education in

computer architecture and digital design, which I will hopefully be able to merge with interesting topics in mathematics to utilize my background (potential math thesis topics coming soon ..?).